

ชุดใบงานรายวิชา BBAIS219
หัวข้อพิเศษในการพัฒนาซอฟต์แวร์

ใบงานที่ 7 รายงานฉบับสมบูรณ์

ระบบขายหน้าร้าน (POS) ตลาดสดทวีโรจน์
พัฒนาด้วยเทคโนโลยี AI Coding Assistant

กลุ่มที่ 1 หัวข้อ AI Coding Assistant

ผู้จัดทำ

1. นายวิทยา รหัสนักศึกษา 66541207018-7
2. นายภคพล วราสินธ์ รหัสนักศึกษา 66541207074-0

ภาคเรียนที่ 1 ปีการศึกษา 2569

สารบัญ

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

1.2 ขอบเขตของโครงการ

1.3 ประโยชน์ที่คาดว่าจะได้รับ

บทที่ 2 วัตถุประสงค์

บทที่ 3 การทบทวนวรรณกรรม

บทที่ 4 วิธีดำเนินงาน

บทที่ 5 ผลการพัฒนา

บทที่ 6 ผลการทดลอง

บทที่ 7 สรุป

บทที่ 8 ข้อเสนอแนะ

เอกสารอ้างอิง

ภาคผนวก ก คู่มือการติดตั้งและใช้งานระบบ

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ตลาดสดหัวโหลเป็นตลาดสดขนาดกลางที่มีพ่อค้าแม่ค้าจำนวนมาก การขายสินค้าในปัจจุบันยังคงใช้การคำนวณราคาด้วยมือหรือเครื่องคิดเลขทั่วไป ในช่วงเวลาที่มีลูกค้าหนาแน่น เช่น ช่วงเช้าและช่วงเย็น ผู้ขายต้องคิดเงินหลายรายการพร้อมกัน ทำให้เกิดความล่าช้า ลูกค้าต้องรอคิวนาน และมีโอกาสเกิดข้อผิดพลาดจากมนุษย์ (Human Error) ทั้งการคำนวณผิด การทอนเงินผิด และการจดบันทึกจำนวนสินค้าคลาดเคลื่อน ส่งผลให้การสรุปยอดขายและการตรวจนับสต็อกสินค้าไม่ตรงกับความเป็นจริง

ในอีกด้านหนึ่ง ร้านค้าขนาดเล็กมักไม่มีงบประมาณจ้างนักพัฒนาซอฟต์แวร์เพื่อสร้างระบบขายหน้าร้าน (Point of Sale: POS) ของตนเอง เทคโนโลยี **AI Coding Assistant** เช่น ChatGPT และ GitHub Copilot ซึ่งสามารถช่วยเขียนโค้ด อธิบายโค้ด และช่วยหาข้อผิดพลาดได้ จึงเป็นเครื่องมือที่น่าสนใจในการลดเวลาและต้นทุนการพัฒนาบบลง ผู้จัดทำจึงเลือกศึกษาเทคโนโลยี AI Coding Assistant และนำมาใช้พัฒนาระบบขายหน้าร้านต้นแบบสำหรับตลาดสดหัวโหล เพื่อแก้ปัญหาทางธุรกิจที่เกิดขึ้นจริง พร้อมทั้งประเมินว่าเทคโนโลยีนี้ช่วยการพัฒนาได้จริงเพียงใดและมีข้อควรระวังอะไรบ้าง

1.2 ขอบเขตของโครงการ

- พัฒนาระบบเป็นเว็บแอปพลิเคชันด้วย HTML, CSS, JavaScript และ PHP เชื่อมต่อฐานข้อมูล MySQL ผ่าน phpMyAdmin (XAMPP)
- ผู้ใช้งานหลักคือพ่อค้าแม่ค้า (Seller) ที่หน้าร้าน
- จำลองการสแกนบาร์โค้ดด้วยการคลิกการ์ดสินค้า (ยังไม่เชื่อมต่อเครื่องสแกนจริง)
- ครอบคลุมการเพิ่ม/ลบสินค้าในตะกร้า คำนวณยอด ตรวจสอบสต็อก ออกใบเสร็จ พิมพ์ใบเสร็จ และตัดสต็อกในฐานข้อมูล
- ไม่ครอบคลุมระบบสมาชิก ระบบเข้าสู่ระบบ (Login) และรายงานยอดขายประจำวัน

1.3 ประโยชน์ที่คาดว่าจะได้รับ

- ลดข้อผิดพลาดในการคำนวณราคาและลดเวลาในการคิดเงินลูกค้าแต่ละราย
- ข้อมูลสต็อกสินค้าในฐานข้อมูลถูกปรับปรุงอัตโนมัติทุกครั้งที่ย้าย
- ลูกค้าได้รับใบเสร็จที่มีรายละเอียดครบถ้วน พร้อมจำนวนเงินเป็นตัวอักษรภาษาไทย
- ผู้จัดทำได้เรียนรู้การใช้ AI Coding Assistant อย่างถูกวิธี ทั้งจุดแข็งและข้อจำกัดของเครื่องมือ

บทที่ 2 วัตถุประสงค์

1. เพื่อศึกษาเทคโนโลยี AI Coding Assistant ทั้งหลักการทำงาน ข้อดี ข้อจำกัด และตัวอย่างการใช้งานจริง
2. เพื่อทบทวนงานวิจัยและกรณีศึกษาที่เกี่ยวข้องกับผลกระทบของ AI Coding Assistant ต่อผลิตภาพ ความปลอดภัย และการเรียนรู้
3. เพื่อออกแบบและพัฒนาระบบขายหน้าร้าน (POS) ต้นแบบสำหรับตลาดสดวิโรจน์ โดยใช้ AI Coding Assistant ช่วยในการพัฒนา
4. เพื่อทดสอบความถูกต้องของระบบ และประเมินผลการใช้งานจากผู้ทดลอง
5. เพื่อสรุปแนวทางการใช้ AI Coding Assistant ในการพัฒนาซอฟต์แวร์อย่างมีประสิทธิภาพและปลอดภัย

บทที่ 3 การทบทวนวรรณกรรม

3.1 AI Coding Assistant คืออะไร

AI Coding Assistant คือเทคโนโลยีปัญญาประดิษฐ์ที่สนับสนุนการพัฒนาซอฟต์แวร์ โดยอาศัยโมเดลภาษาขนาดใหญ่ (Large Language Model: LLM) ที่ถูกฝึกด้วยข้อความและซอร์สโค้ดจำนวนมาก ทำให้สามารถรับคำสั่งเป็นภาษาธรรมชาติ (Prompt) แล้วสร้างโค้ด แนะนำคำสั่ง อธิบายโค้ด ตรวจสอบข้อผิดพลาด และเสนอวิธีแก้ไขได้ (GitHub, 2021) เครื่องมือที่เป็นที่รู้จัก ได้แก่

- **ChatGPT** — ผู้ช่วยแบบสนทนา ใช้เขียนโค้ด อธิบายการทำงาน วิเคราะห์ Bug และช่วยออกแบบระบบ (OpenAI, 2026a)
- **GitHub Copilot** — ทำงานร่วมกับ Visual Studio Code และ IDE อื่น แนะนำโค้ดอัตโนมัติจากบริบทของไฟล์ (GitHub, 2026)
- **OpenAI Codex / Coding Agent** — วิเคราะห์ทั้งโปรเจกต์ แก้ไขหลายไฟล์ รันคำสั่งและทดสอบโปรแกรมได้เอง (OpenAI, 2026b)

แนวโน้มล่าสุดคือเครื่องมือกำลังพัฒนาจาก "การเติมโค้ดที่ละบรรทัด" ไปสู่ "Agent" ที่ทำงานหลายขั้นตอนได้อัตโนมัติ ผู้พัฒนาจึงต้องเปลี่ยนบทบาทเป็นผู้กำหนดโจทย์ ผู้ตรวจสอบ และผู้ทดสอบผลงานของ AI มากขึ้น (Cursor, 2026; GitHub Changelog, 2026)

3.2 ผลกระทบต่อผลิตภาพของนักพัฒนา

งานวิจัยแบบมีกลุ่มควบคุมของ Peng et al. (2023) พบว่านักพัฒนาที่ใช้ GitHub Copilot เขียน HTTP Server ด้วย JavaScript เสร็จเร็วกว่ากลุ่มควบคุมถึง 55.8% โดยผู้มีประสบการณ์น้อยได้รับประโยชน์มากที่สุด ขณะที่ Ziegler et al. (2024) พบว่าอัตราการยอมรับคำแนะนำของ AI เป็นตัวชี้วัดที่ทำนายผลิตภาพที่ผู้ใช้รับรู้ได้ดีที่สุด

อย่างไรก็ตาม งานวิจัยบางชิ้นให้ผลที่ต่างออกไป Stray et al. (2026) วิเคราะห์ commit กว่า 26,000 รายการในองค์กรจริง 2 ปี ไม่พบการเปลี่ยนแปลงที่มีนัยสำคัญทางสถิติหลังเริ่มใช้ Copilot และ Vaithilingam et al. (2022) พบว่าผู้ใช้ "รู้สึก" ว่าประหยัดเวลา แต่เวลาทำงานจริงไม่ต่างกันอย่างมีนัยสำคัญ เนื่องจากต้องใช้เวลาตรวจสอบโค้ดที่ AI สร้าง ข้อสรุปคือ **ประโยชน์ของ AI ขึ้นกับลักษณะงานและทักษะการตรวจสอบของผู้ใช้** สอดคล้องกับ McKinsey & Company (2023) ที่ระบุว่า AI ช่วยได้มากที่สุดในช่วงเริ่มต้นงานและงานที่ทำซ้ำ

3.3 ความปลอดภัยของโค้ดที่ AI สร้าง

Pearce et al. (2022) ทดสอบ Copilot กับ 89 สถานการณ์ตาม MITRE CWE Top 25 พบว่าประมาณ 40% ของโปรแกรมที่สร้างมีช่องโหว่ Fu et al. (2024) พบช่องโหว่ในโค้ดจาก AI ที่ใช้ในโปรเจกต์ GitHub จริงราว 30% ของโค้ด Python และ 24% ของโค้ด JavaScript ส่วน Asare et al. (2023) และ

Sandoval et al. (2023) พบว่า AI ไม่ได้แย่กว่ามนุษย์อย่างมีนัยสำคัญ แต่ก็ไม่ได้ช่วยลดความเสี่ยงลง ดังนั้น การตรวจสอบความปลอดภัยของโค้ดเป็นขั้นตอนบังคับ ไม่ใช่ทางเลือก เมื่อนำ AI มาใช้ในงานจริง

3.4 การใช้ AI ในการเรียนรู้การเขียนโปรแกรม

Sun et al. (2024) พบว่านักศึกษาที่ใช้ ChatGPT ช่วยเขียนโปรแกรมมีพฤติกรรมการติบักและขอ feedback มากขึ้น แต่ต้องระวังโค้ดที่ไม่ถูกต้อง และ Chen et al. (2024) พบว่าการให้ผู้เรียน "สอน" AI ช่วยให้เกิดความเข้าใจเชิงลึกได้ดีกว่าการรับคำตอบสำเร็จรูป

3.5 สถานการณ์การใช้งานในอุตสาหกรรม

ผลสำรวจ Stack Overflow (2025) จากนักพัฒนาที่ 49,000 คน พบว่า 84% ใช้หรือวางแผนจะใช้ AI Coding Assistant แต่มีผู้ไม่ไว้วางใจความถูกต้องของผลลัพธ์ (46%) มากกว่าผู้ไว้วางใจ (33%) สอดคล้องกับข้อมูลระยะยาวของ Our World in Data (Mathieu et al., 2023) ที่แสดงว่าสัดส่วนผู้ใช้เพิ่มขึ้นต่อเนื่องทุกปี

3.6 ข้อสรุปจากการทบทวนวรรณกรรมที่นำมาใช้ในโครงการ

ข้อค้นพบจากงานวิจัย	สิ่งที่โครงการนำมาปฏิบัติ
AI ช่วยเริ่มต้นงานได้เร็ว (Peng et al., 2023; McKinsey, 2023)	ใช้ ChatGPT ร่างโครงสร้างระบบ แยกไฟล์ และสร้างโค้ดตั้งต้นของแต่ละโมดูล
ผู้ใช้อาจตรวจสอบโค้ดเอง (Vaithilingam et al., 2022)	อ่านและทดสอบโค้ดทุกส่วนก่อนใช้ พร้อมเขียนชุดทดสอบอัตโนมัติ (tests/test_logic.mjs)
โค้ดจาก AI มีช่องโหว่ได้ (Pearce et al., 2022; Fu et al., 2024)	ใช้ Prepared Statement ป้องกัน SQL Injection และ Transaction ป้องกันข้อมูลสต็อกเสียหาย
Prompt ที่ชัดเจนให้ผลดีกว่า (Cursor, 2026)	ระบุภาษา เงื่อนไข และตัวอย่างผลลัพธ์ใน Prompt ทุกครั้ง

บทที่ 4 วิธีดำเนินงาน

4.1 ขั้นตอนการดำเนินงาน

โครงการดำเนินการตามลำดับใบงานของรายวิชา ดังตารางต่อไปนี้

ขั้นตอน	กิจกรรม	ผลลัพธ์
1. สำรวจเทคโนโลยี	ศึกษา AI Coding Assistant ข้อดี ข้อจำกัด ตัวอย่างการใช้งาน	ใบงานที่ 1
2. ทบทวนวรรณกรรม	สรุปบทความวิชาการ 10 เรื่อง และบทความเว็บไซต์ 10 แหล่ง	ใบงานที่ 2
3. ทดลองใช้เครื่องมือ	ใช้ ChatGPT สร้างโปรแกรมคำนวณราคาสินค้า (Python) และทดสอบผล	ใบงานที่ 3
4. ออกแบบต้นแบบ	กำหนด Business Problem, Functional Requirements, Use Case, หน้าจอ, Workflow	ใบงานที่ 4
5. พัฒนาระบบ	พัฒนาเว็บ POS ด้วย HTML/CSS/JS/PHP + MySQL โดยใช้ AI ช่วย	ใบงานที่ 5
6. ประเมินผล	ให้เพื่อนทดลองใช้งานและเก็บข้อเสนอแนะ 5 ด้าน	ใบงานที่ 6
7. ปรับปรุงและทดสอบ	แก้ไขตามข้อเสนอแนะ ทดสอบระบบและแก้ Bug ที่พบ	ใบงานที่ 7

4.2 เครื่องมือที่ใช้

เครื่องมือ	หน้าที่
ChatGPT (AI Coding Assistant)	ช่วยออกแบบโครงสร้าง เขียนโค้ด อธิบายโค้ด และวิเคราะห์ข้อผิดพลาด
Visual Studio Code	เขียนและแก้ไขซอร์สโค้ด
XAMPP (Apache + PHP + MySQL)	จำลองเว็บเซิร์ฟเวอร์และฐานข้อมูลบนเครื่อง (Localhost)
phpMyAdmin	สร้างและจัดการฐานข้อมูล thawirot_pos
Google Chrome + DevTools	ทดสอบหน้าเว็บ ตรวจสอบ Network และ Console
Node.js	รันชุดทดสอบตรรกะการคำนวณอัตโนมัติ

4.3 กระบวนการพัฒนาด้วย AI Coding Assistant

ผู้จัดทำใช้ AI ในลักษณะ "ผู้ช่วยคู่คิด" (Pair Programmer) ไม่ใช่การคัดลอกโค้ดไปใช้ทันที โดยทำงานเป็นวงรอบดังนี้

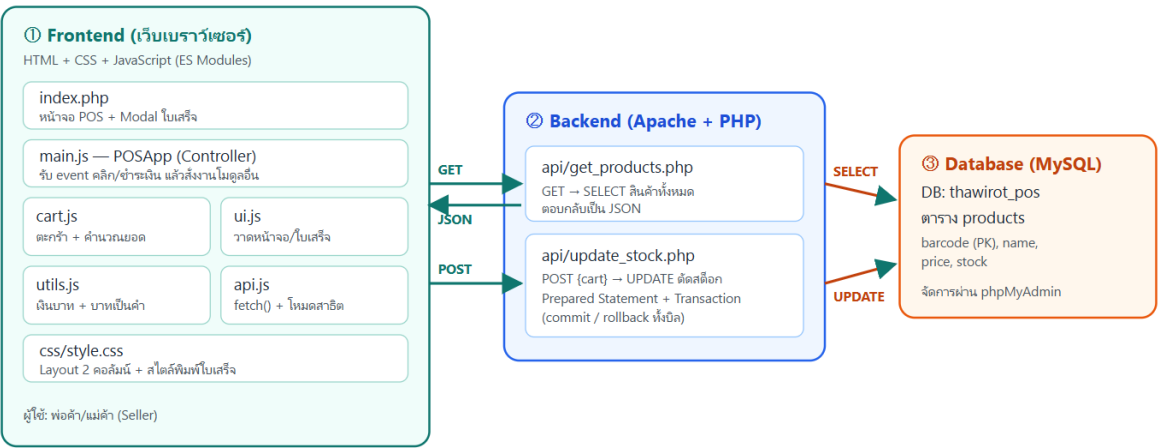
- กำหนดโจทย์ (Prompt) ระบุภาษา ไฟล์ที่เกี่ยวข้อง เงื่อนไข และตัวอย่างผลลัพธ์ให้ชัดเจน
- รับโค้ดและอ่านทำความเข้าใจ ขอให้ AI อธิบายโค้ดทีละส่วน จนผู้จัดทำอธิบายเองได้
- ทดสอบ รันในเบราร์เซิร์ฟเวอร์และด้วยชุดทดสอบ เทียบผลกับค่าที่คำนวณด้วยมือ
- แก้ไข เมื่อพบข้อผิดพลาด ส่งอาการของ Bug กลับไปให้ AI วิเคราะห์ หรือแก้ด้วยตนเอง แล้วทดสอบซ้ำ

ตัวอย่าง Prompt ที่ใช้ในการพัฒนา

- "ช่วยแยกโค้ด JavaScript ของระบบ POS ออกเป็นโมดูล: ตะกร้า (cart), หน้าจอ (ui), เรียก API (api) และตัวควบคุมหลัก (main) โดยใช้ ES Modules"
- "เขียน PHP รับ JSON ตะกร้าสินค้า แล้วตัดสต็อกใน MySQL ใช้ Prepared Statement และ Transaction ถ้ารายการใดผิดพลาดให้ยกเลิกทั้งบิล"
- "เขียนฟังก์ชันแปลงจำนวนเงินเป็นตัวอักษรภาษาไทย เช่น 215 เป็น สองร้อยสิบห้าบาทถ้วน รอรับสตางค์ และยกตัวอย่างกรณีทดสอบ"

4.4 สถาปัตยกรรมระบบ

ระบบออกแบบเป็นสถาปัตยกรรม 3 ชั้น (3-Tier Architecture) แยกหน้าที่ชัดเจน ดังภาพที่ 1



ภาพที่ 1 สถาปัตยกรรมระบบ 3 ชั้น

- **ชั้นที่ 1 Frontend** ทำงานบนเบราว์เซอร์ แบ่ง JavaScript เป็น 5 โมดูลตามหลัก Separation of Concerns: main.js (ตัวควบคุม), cart.js (ข้อมูลตะกร้า), ui.js (แสดงผล), api.js (ติดต่อเซิร์ฟเวอร์), utils.js (ฟังก์ชันช่วย)
- **ชั้นที่ 2 Backend** PHP 2 ไฟล์ทำหน้าที่เป็น REST-style API รับ-ส่งข้อมูลรูปแบบ JSON
- **ชั้นที่ 3 Database** MySQL เก็บข้อมูลสินค้าและสต็อกในตาราง products

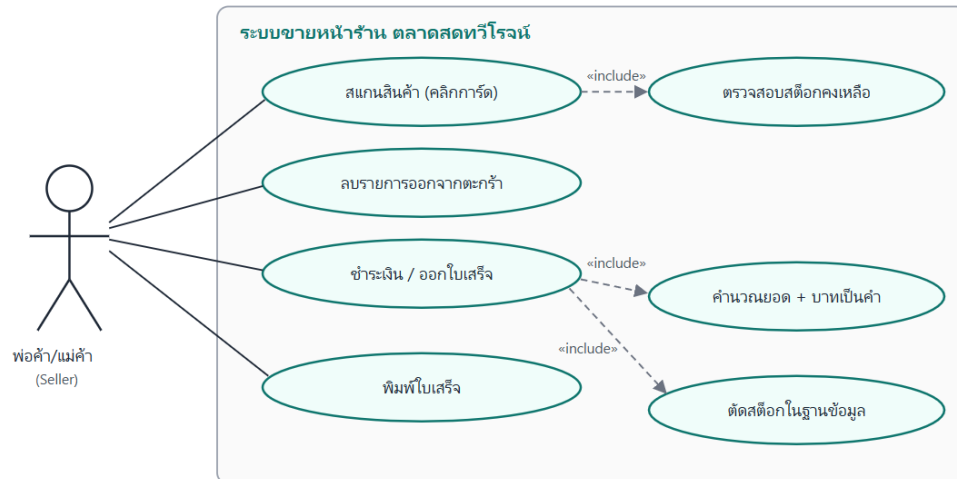
4.5 การออกแบบฐานข้อมูล

ตารางที่ 1 โครงสร้างตาราง products (ฐานข้อมูล thawirot_pos)

ฟิลด์	ชนิดข้อมูล	คำอธิบาย
barcode	VARCHAR(13) PRIMARY KEY	รหัสบาร์โค้ด EAN-13 (ไม่ซ้ำกัน)
name	VARCHAR(100)	ชื่อสินค้า
price	DECIMAL(10,2)	ราคาต่อหน่วย (บาท) — ใช้ DECIMAL เพื่อความแม่นยำของเงิน

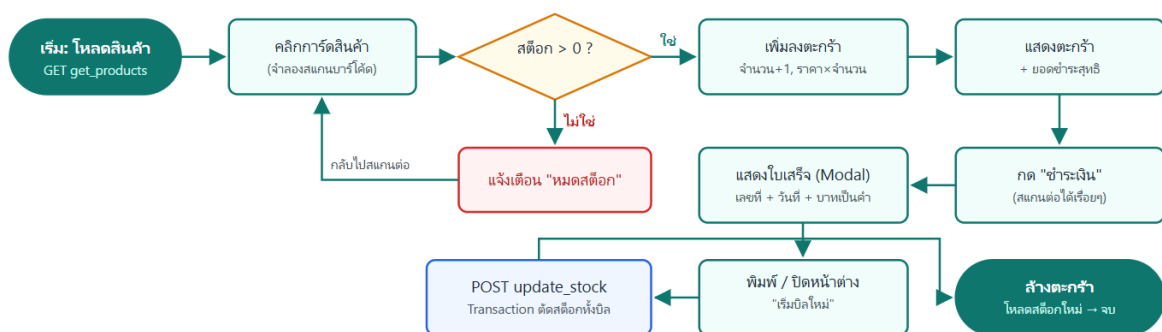
stock	INT, CHECK (stock >= 0)	จำนวนคงเหลือ ห้ามติดลบ
-------	-------------------------	------------------------

4.6 Use Case Diagram



ภาพที่ 2 Use Case Diagram ของระบบ

4.7 Workflow การทำงาน

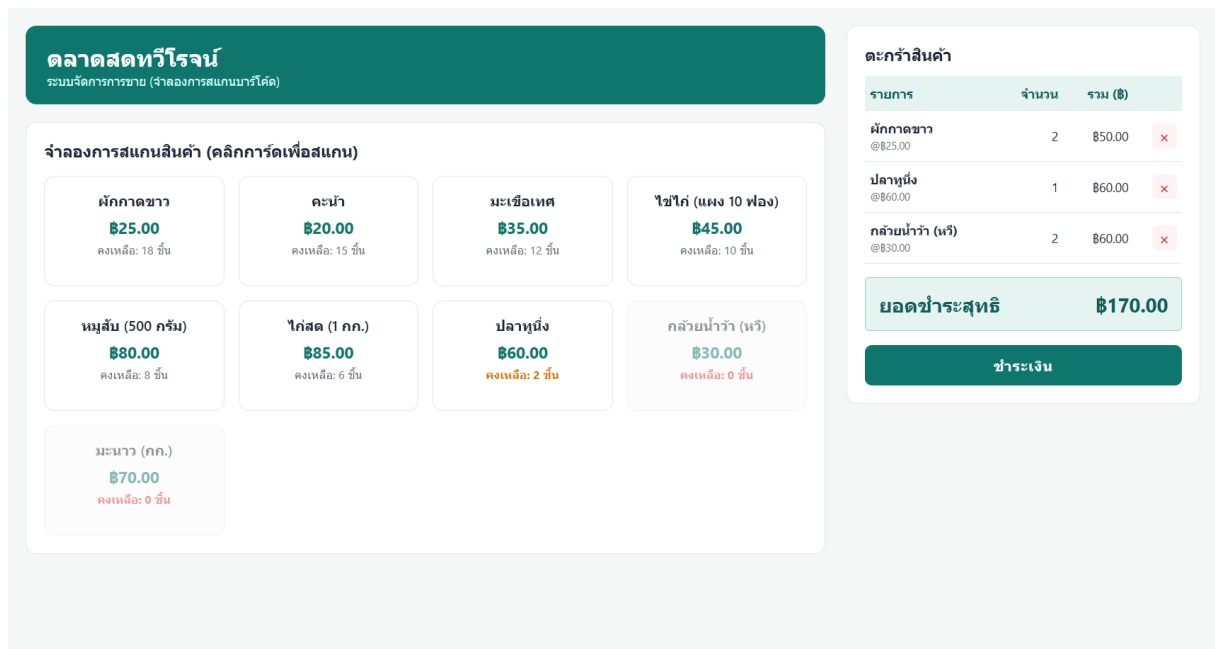


ภาพที่ 3 Workflow การขายสินค้า 1 บิล

บทที่ 5 ผลการพัฒนา

5.1 หน้าจอหลักของระบบ

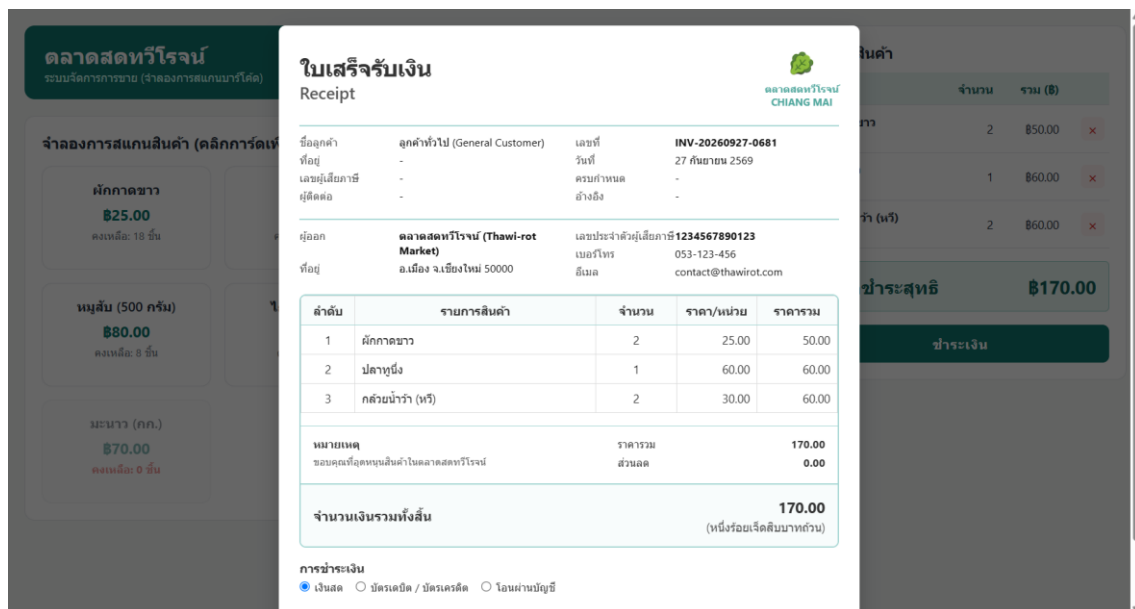
หน้าจอหลักแบ่งเป็น 2 ฝั่ง ฝั่งซ้ายคือการดิสสินค้าสำหรับจำลองการสแกนบาร์โค้ด แสดงชื่อ ราคา และจำนวนคงเหลือ สินค้าที่ใกล้หมด (ไม่เกิน 3 ชิ้น) แสดงตัวเลขสีส้ม และสินค้าที่หมดแล้วจะเป็นสีจางและกดไม่ได้ ฝั่งขวาคือตะกร้าสินค้าที่แสดงรายการ จำนวน ราคารวม และยอดชำระสุทธิ



ภาพที่ 4 หน้าจอหลักระหว่างสแกนสินค้า

5.2 ใบเสร็จรับเงิน

เมื่อกดปุ่ม "ชำระเงิน" ระบบจะแสดงใบเสร็จแบบ Pop-up ประกอบด้วยเลขที่ใบเสร็จ วันที่ (ปี พุทธศักราช) ข้อมูลผู้ออก รายการสินค้า ยอดรวม และจำนวนเงินเป็นตัวอักษรภาษาไทย สามารถกด "พิมพ์ใบเสร็จ" ได้ โดยระบบซ่อนส่วนอื่นของหน้าจอออกขณะพิมพ์



ภาพที่ 5 ใบเสร็จรับเงิน

5.3 ฟังก์ชันที่พัฒนาได้

ฟังก์ชัน	รายละเอียด	ไฟล์ที่เกี่ยวข้อง
โหลดข้อมูลสินค้า	ดึงรายการสินค้าจาก MySQL ผ่าน fetch() ได้ข้อมูล JSON	api.js, get_products.php
จำลองสแกนบาร์โค้ด	คลิกการ์ดสินค้าเพื่อเพิ่มลงตะกร้า สินค้าเดิมจะรวมจำนวนเป็นบรรทัดเดียว	main.js, cart.js
ตรวจสอบสต็อก	ห้ามเพิ่มสินค้าเกินจำนวนคงเหลือ แจ้งเตือนเมื่อหมด	main.js, cart.js
ลบรายการ	กดปุ่ม x เพื่อลบ และคืนจำนวนสต็อกบนหน้าจอ	cart.js
คำนวณยอด	ราคา x จำนวน ต่อรายการ และรวมเป็นยอดชำระสุทธิ	cart.js, ui.js
ออก/พิมพ์ใบเสร็จ	เลขที่ใบเสร็จ วันที่ไทย จำนวนเงินเป็นตัวอักษร พิมพ์ได้	ui.js, utils.js
ตัดสต็อกในฐานข้อมูล	ส่งตะกร้าทั้งบิลไปตัดสต็อกใน Transaction	update_stock.php
โหมดสาธิต	หากเชื่อมต่อฐานข้อมูลไม่ได้ ใช้ข้อมูลตัวอย่างแทนพร้อมแถบแจ้งเตือน	api.js

5.4 กลไกทางเทคนิคที่สำคัญ

(1) Transaction และ Prepared Statement ในการตัดสต็อก ไฟล์ update_stock.php เริ่ม Transaction ก่อนตัดสต็อกทุกรายการในบิล หากรายการใดล้มเหลว จะ rollback ทั้งบิล ข้อมูลจึงไม่ค้างครึ่งๆ กลางๆ และใช้ Prepared Statement (เครื่องหมาย ?) แยกคำสั่ง SQL ออกจากข้อมูล เพื่อป้องกัน SQL Injection

```
$conn->begin_transaction();
$sql = "UPDATE products SET stock = stock - ? WHERE barcode = ? AND stock >= ?";
$stmt = $conn->prepare($sql);
```

```
foreach ($data['cart'] as $item) {
    $stmt->bind_param("isi", $quantity, $barcode, $quantity);
    $stmt->execute();
    if ($stmt->affected_rows === 0) throw new Exception("สต็อกไม่พอ...");
}
$conn->commit(); // ถ้ามี Exception จะไปที่ catch แล้ว $conn->rollback();
```

(2) การตรวจสอบสต็อก 3 ชั้น ชั้นแรกที่หน้าจอ (cart.js ไม่ให้เพิ่มเกินสต็อก) ชั้นที่สองที่ PHP (เงื่อนไข stock >= ? ใน SQL) และชั้นที่สามที่ฐานข้อมูล (CHECK stock >= 0) เพื่อรองรับกรณีเปิดหลายเครื่องขายสินค้าขึ้นสุดท้ายพร้อมกัน

(3) การแยกโมดูลด้วย ES Modules แต่ละไฟล์ export เฉพาะสิ่งที่จำเป็น และ main.js เป็นตัวประสานงาน ทำให้แก้ไขส่วนใดส่วนหนึ่งได้โดยไม่กระทบส่วนอื่น และทดสอบ cart.js กับ utils.js แยกได้โดยไม่ต้องเปิดเบราว์เซอร์

(4) การแปลงจำนวนเงินเป็นคำอ่านภาษาไทย ฟังก์ชัน ThaiBahtText อ่านตัวเลขทีละหลักพร้อมหน่วย (สิบ ร้อย พัน หมื่น แสน) วนทุก 6 หลักเป็น "ล้าน" และจัดการกรณีพิเศษของภาษาไทย ได้แก่ "สิบ" (ไม่ใช่หนึ่งสิบ) "ยี่สิบ" (ไม่ใช่สองสิบ) และ "เอ็ด" (หลักหน่วยเป็น 1 เมื่อมีหลักอื่นนำหน้า)

5.5 การปรับปรุงหลังการประเมิน (ใบงานที่ 6)

จากข้อเสนอแนะว่าหน้าจอ "ขาดความสบายตาและขาดการใช้สีชั้นนำการใช้งาน" ผู้จัดทำใช้ AI Coding Assistant ช่วยตรวจสอบไฟล์ CSS และพบว่าไฟล์ style.css เรียกใช้ตัวแปรสี (เช่น --primary, --border, --danger) และคลาสจัดวาง (เช่น .pos-container, .btn, .alert, .hidden) ที่ไม่เคยถูกประกาศไว้ ทำให้ตะกร้าไปอยู่ใต้รายการสินค้า ปุ่มไม่มีสี และสีเตือนสต็อกใกล้หมดไม่แสดง จึงปรับปรุงดังนี้

- ประกาศชุดสีหลักโทนเขียวอมฟ้า (Teal) สื่อถึงตลาดสด และสีแดง/ส้มสำหรับการแจ้งเตือน
- จัดวางแบบ 2 คอลัมน์ ตะกร้าอยู่ด้านขวาและติดอยู่กับที่เวลาเลื่อนหน้าจอ รองรับจอแท็บเล็ต (Responsive)
- ปุ่มชำระเงินขนาดใหญ่เพิ่มความกว้าง ปุ่มลบรายการสีแดง สต็อกใกล้หมดสีส้ม สินค้าหมดสีจางและกดไม่ได้

บทที่ 6 ผลการทดลอง

6.1 การทดสอบการทำงานของระบบ (System Test)

ทดสอบบนเครื่องที่ติดตั้ง XAMPP 8.2.12 (Apache, PHP 8.2, MariaDB 10.4) ผ่านเว็บเบราว์เซอร์ Google Chrome ตาม Use Case หลัก และทดสอบ API ด้วยการส่งคำขอ HTTP โดยตรง (curl) ใช้ข้อมูลสินค้าตัวอย่างจากไฟล์ database.sql ได้ผลดังตารางที่ 2

ตารางที่ 2 ผลการทดสอบการทำงานของระบบ

รหัส	กรณีทดสอบ	ผลที่คาดหวัง	ผล
TC-01	เปิดหน้าระบบ	แสดงการ์ดสินค้า 9 รายการ พร้อมราคาและสต็อก	ผ่าน
TC-02	คลิกผักกาดขาว 2 ครั้ง	ตะกร้าแสดง 1 บรรทัด จำนวน 2 รวม ฿50.00 สต็อกหน้าจอ 20→18	ผ่าน
TC-03	สแกนผักกาดขาว×2 ไข่ไก่×1 ปลาหูช้าง×1 กุ้ง×2	ยอดชำระสุทธิ ฿215.00	ผ่าน
TC-04	สแกนกล้วยน้ำว้าจนหมด (2 หวี)	การ์ดเปลี่ยนเป็นสีจาง กดเพิ่มไม่ได้	ผ่าน
TC-05	สินค้าคงเหลือ ≤ 3	ตัวเลขคงเหลือเป็นสีส้ม	ผ่าน
TC-06	กด × ลบรายการ	รายการหายจากตะกร้า ยอดลดลง สต็อกหน้าจอคืนค่า	ผ่าน
TC-07	ตะกร้าว่าง	ปุ่มชำระเงินกดไม่ได้	ผ่าน
TC-08	กดชำระเงิน (ยอด 215)	ใบเสร็จแสดง 215.00 และ "(สงวนลิขสิทธิ์ห้าบาทถ้วน)" พร้อมเลขที่และวันที่	ผ่าน
TC-09	กดปิดหน้าต่าง/เริ่มบิลใหม่	สต็อกในฐานข้อมูลถูกลบคืน ตะกร้าว่าง หน้าจอโหลดสต็อกใหม่ (ผักกาดขาว 18)	ผ่าน
TC-10	ตั้งสต็อกคื่นหีใน DB เหลือ 1 แล้วขาย 3 ชิ้น	ไม่ตัดสต็อก ใบเสร็จปิด แจ้ง "สต็อกไม่พอ" ตะกร้ายังอยู่ สต็อกใน DB ยังเป็น 1	ผ่าน
TC-11	บิลมี ผักกาดขาว 1 + คื่นหี 3 (คื่นหีไม่พอ)	Rollback ทั้งบิล ผักกาดขาวที่ตัดไปแล้วถูกคืน (ยังเป็น 20)	ผ่าน
TC-12	ส่ง barcode = ' OR 1=1 -- ไปที่ API	ไม่มีสินค้าใดถูกตัดสต็อก (Prepared Statement กัน SQL Injection)	ผ่าน
TC-13	ส่งจำนวน 0 / ตะกร้าว่าง ไปที่ API	ปฏิเสธคำขอพร้อมข้อความแจ้งเหตุผล	ผ่าน
TC-14	ส่ง UPDATE สต็อกเป็น -1 ตรงๆ ในฐานข้อมูล	ฐานข้อมูลปฏิเสธ (CONSTRAINT chk_stock failed)	ผ่าน
TC-15	ฐานข้อมูลใช้งานไม่ได้	เข้าโหมดสาธิต แสดงแถบสีเหลือง ยังขายได้	ผ่าน

TC-10 ถึง TC-14 เป็นกรณีที่เพิ่มหลังพบ Bug สต็อกติดลบ (ดูหัวข้อ 6.3) ผลการทดสอบผ่านทุกกรณี (15/15)

ตลาดสดหัวโขน

ระบบจัดการการขาย (จำลองการสแกนบาร์โค้ด)

จำลองการสแกนสินค้า (คลิกการ์ดเพื่อสแกน)

ข้อผิดพลาดในการตัดสต็อก: สต็อกไม่พอหรือไม่มีสินค้าบาร์โค้ด: 8850001000028

ผักกาดขาว

฿25.00

คงเหลือ: 18 ชิ้น

คะน้า

฿20.00

คงเหลือ: 12 ชิ้น

มะเขือเทศ

฿35.00

คงเหลือ: 12 ชิ้น

ไข่ไก่ (แผง 10 ฟอง)

฿45.00

คงเหลือ: 10 ชิ้น

หนุ่ย (500 กรัม)

฿80.00

คงเหลือ: 8 ชิ้น

ไก่สด (1 กก.)

฿85.00

คงเหลือ: 6 ชิ้น

ปลาทูน่า

฿60.00

คงเหลือ: 2 ชิ้น

กล้วยน้ำว้า (หวี)

฿30.00

คงเหลือ: 0 ชิ้น

มะนาว (กก.)

฿70.00

คงเหลือ: 0 ชิ้น

ตะกร้าสินค้า

รายการ	จำนวน	รวม (฿)
คะน้า @฿20.00	3	฿60.00

ยอดชำระสุทธิ

฿60.00

ชำระเงิน

ภาพที่ 6 ผลทดสอบ TC-10 ระบบปฏิเสธการตัดสต็อกเมื่อสินค้าไม่พอ

6.2 การทดสอบตรรกะการคำนวณอัตโนมัติ (Unit Test)

เขียนชุดทดสอบ tests/test_logic.mjs รันด้วยคำสั่ง node tests/test_logic.mjs ทดสอบโมดูล cart.js 6 กรณี และฟังก์ชัน ThaiBahtText 8 กรณี ผลการทดสอบ ผ่านทั้งหมด 14/14 กรณี ตัวอย่างกรณีทดสอบ ได้แก่ ยอดรวม $(30 \times 2) + (45 \times 1) = 105$ ตรงกับผลการทดลองในใบงานที่ 3, 101 → "หนึ่งร้อยเอ็ดบาทถ้วน", 1,250.50 → "หนึ่งพันสองร้อยห้าสิบบาทห้าสิบสตางค์" และ 2,500,000 → "สองล้านห้าแสนบาทถ้วน"

6.3 ข้อผิดพลาด (Bug) ที่พบและการแก้ไข

ตารางที่ 3 Bug ที่พบระหว่างการทดสอบ

ปัญหาที่พบ	สาเหตุ	การแก้ไข
หน้าจอไม่มีสี ตะกร้าอยู่ใต้สินค้า	CSS เรียกใช้ตัวแปรสีและคลาสที่ไม่ได้ประกาศ	เพิ่ม :root และคลาส Layout/ปุ่ม/แจ้งเตือน
สต็อกในฐานข้อมูลติดลบได้	SQL ตัดสต็อกโดยไม่ตรวจสอบว่าพอหรือไม่	เพิ่มเงื่อนไข stock >= ? ตรวจสอบ affected_rows และ CHECK ในตาราง
ยอด 1,000,000 บาท อ่านว่า "หนึ่งบาทถ้วน"	ฟังก์ชันไม่รองรับหลักล้าน (หน่วยวนทุก 6 หลัก)	เขียนฟังก์ชัน readThaiNumber ใหม่เติม "ล้าน" ทุก 6 หลัก
0.01 บาท อ่านว่า "เอ็ดสตางค์"	เงื่อนไข "เอ็ด" ดูแค่ความยาวตัวเลข	ใช้ "เอ็ด" เฉพาะเมื่อมีหลักอื่นที่ไม่ใช่ 0 นำหน้า
ข้อความแจ้งเตือนตัดสต็อกไม่สำเร็จมองไม่เห็น	ข้อความอยู่หลังหน้าต่างโอบีเสิร์จ	ปิดโอบีเสิร์จก่อน แล้วแสดงข้อความ ตะกร้ายังอยู่ให้แก้ไข
ย้ายเครื่องแล้วระบบรันไม่ได้	ไม่มีไฟล์สร้างฐานข้อมูล	เพิ่ม database.sql และคู่มือ README.md

ข้อค้นพบนี้สอดคล้องกับงานวิจัยในบทที่ 3 ว่าโค้ดที่ได้จาก AI ทำงานได้ในกรณีทั่วไป แต่อาจพลาดในกรณีขอบ (Edge Case) เช่น จำนวนเงินหลักล้าน หรือการขายพร้อมกันหลายเครื่อง จึงต้องมีการทดสอบอย่างเป็นระบบเสมอ

6.4 ผลการประเมินจากผู้ทดลอง

ให้เพื่อนทดลองใช้งานระบบและประเมิน 5 ด้าน สรุปผลได้ดังตารางที่ 4

ตารางที่ 4 ผลการประเมินจากผู้ทดลอง (ใบงานที่ 6)

ด้าน	ผลการประเมิน
ความใช้งานง่าย	ใช้งานได้ง่าย ไม่ซับซ้อน เรียนรู้วิธีใช้งานได้รวดเร็ว
ความถูกต้อง	ระบบคำนวณและการสแกนบาร์โค้ด (จำลอง) ทำงานได้ถูกต้อง
ความเร็ว	ขั้นตอนไม่มาก ไม่มีการกระตุกหรือโหลดช้า
ความสวยงาม	ขาดความสบายตาและขาดการใช้สีขึ้นำการใช้งาน (ได้ปรับปรุงแล้วตามหัวข้อ 5.5)
ความเป็นประโยชน์	ลดความผิดพลาดและลดเวลาในการคิดคำนวณได้จริง

บทที่ 7 สรุป

โครงการนี้ได้ศึกษาเทคโนโลยี AI Coding Assistant และนำมาใช้พัฒนาระบบขายหน้าร้าน (POS) ต้นแบบสำหรับตลาดสดทีวีโรจน์ ระบบสามารถจำลองการสแกนบาร์โค้ด คำนวณราคาอัตโนมัติ ตรวจสอบสต็อก ออกและพิมพ์ใบเสร็จพร้อมจำนวนเงินเป็นตัวอักษรภาษาไทย และตัดสต็อกในฐานข้อมูล MySQL อย่างปลอดภัยด้วย Transaction และ Prepared Statement ผลการทดสอบระบบผ่านทุกกรณีทดสอบ และผู้ใช้ทดลองเห็นว่าระบบใช้งานง่าย ถูกต้อง รวดเร็ว และช่วยลดความผิดพลาดในการคิดเงินได้จริง ซึ่งตอบโจทย์ปัญหาทางธุรกิจที่ตั้งไว้

ในด้านเทคโนโลยี AI Coding Assistant ช่วยให้ผู้จัดทำเริ่มต้นพัฒนาได้เร็ว ช่วยแบ่งโครงสร้างโค้ดเป็นโมดูล และช่วยอธิบายแนวคิดที่ไม่คุ้นเคย เช่น Transaction และ ES Modules อย่างไรก็ตาม การทดสอบพบ Bug ที่มาจากกรณีขอบหลายจุด ยืนยันข้อค้นพบจากงานวิจัยว่า **AI เป็นผู้ช่วยที่ดี แต่ผู้พัฒนาต้องเข้าใจโค้ด ทดสอบ และรับผิดชอบผลลัพธ์เอง** บทเรียนสำคัญที่ได้รับ คือ

- Prompt ที่ระบุเงื่อนไขและตัวอย่างชัดเจน ให้โค้ดที่ตรงความต้องการมากกว่า
- ต้องอ่านและอธิบายโค้ดที่ AI สร้างได้ด้วยตนเองก่อนนำไปใช้
- การเขียนชุดทดสอบช่วยจับข้อผิดพลาดที่มองไม่เห็นจากการทดลองใช้งานทั่วไป
- เรื่องความปลอดภัยและความถูกต้องของข้อมูล (เช่น สต็อกติดลบ) ต้องตรวจสอบเป็นพิเศษ

บทที่ 8 ข้อเสนอแนะ

8.1 ข้อจำกัดของระบบในปัจจุบัน

- ยังจำลองการสแกนด้วยการคลิก ยังไม่รองรับเครื่องสแกนบาร์โค้ดจริง
- ข้อมูลลูกค้าและร้านค้าในใบเสร็จยังเป็นค่าคงที่ ไม่ได้ดึงจากฐานข้อมูล
- ยังไม่บันทึกประวัติการขาย (ตาราง sales) เลขที่ใบเสร็จจึงสุ่มขึ้นมาและอาจซ้ำได้
- ระบบตัดสต็อกเมื่อกด "ปิดหน้าต่าง/เริ่มบิลใหม่" หากปิดเบราร์เซอร์ไปก่อน สต็อกจะไม่ถูกตัด
- ยังไม่มีระบบเข้าสู่ระบบ (Login) และเชื่อมต่อฐานข้อมูลด้วยบัญชี root ซึ่งไม่เหมาะกับการใช้งานจริง
- JavaScript คำนวณเงินด้วยเลขทศนิยมแบบ floating point หากราคามีสตางค์อาจคลาดเคลื่อนเล็กน้อย

8.2 แนวทางการพัฒนาต่อ

- รองรับเครื่องสแกนบาร์โค้ด USB (ทำงานเหมือนคีย์บอร์ด) โดยเพิ่มช่องรับรหัสและกด Enter อัตโนมัติ
- เพิ่มตาราง sales และ sale_items บันทึกทุกบิล เพื่อทำรายงานสรุปยอดขายประจำวัน และสร้างเลขที่ใบเสร็จแบบเรียงลำดับ
- เพิ่มระบบรับเงินและคำนวณเงินทอน ระบบพักบิล และระบบยกเลิกบิล/คืนสินค้า
- ย้ายการตัดสต็อกไปทำตอนกด "ชำระเงิน" และเพิ่มระบบ Login แยกสิทธิ์ผู้ขายกับผู้ดูแล
- เก็บค่าการเชื่อมต่อฐานข้อมูลไว้ในไฟล์ตั้งค่าแยก และสร้างบัญชีฐานข้อมูลที่มีสิทธิ์จำกัด
- คำนวณเงินเป็นหน่วยสตางค์ (จำนวนเต็ม) และเพิ่มการชำระผ่าน QR พร้อมเพย์
- นำ AI Coding Assistant แบบ Agent มาช่วยเขียนชุดทดสอบและตรวจสอบความปลอดภัยของโค้ดอย่างต่อเนื่อง

เอกสารอ้างอิง

- Asare, O., Nagappan, M., & Asokan, N. (2023). Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code? *Empirical Software Engineering*, 28, 129.
<https://doi.org/10.1007/s10664-023-10380-1>
- Chen, A., Wei, Y., Le, H., & Zhang, Y. (2024). Learning-by-teaching with ChatGPT: The effect of teachable ChatGPT agent on programming education. *arXiv:2412.15226*.
<https://arxiv.org/abs/2412.15226>
- Cursor. (2026, January 9). Best practices for coding with agents. <https://cursor.com/blog/agent-best-practices>
- Fu, Y., Liang, P., Tahir, A., Li, Z., Shahin, M., & Yu, J. (2024). Security weaknesses of Copilot-generated code in GitHub projects: An empirical study. *ACM Transactions on Software Engineering and Methodology*. <https://arxiv.org/abs/2310.02059>
- GitHub. (2021, June 29). Introducing GitHub Copilot: Your AI pair programmer. *The GitHub Blog*.
<https://github.blog/2021-06-29-introducing-github-copilot-ai-pair-programmer/>
- GitHub. (2026). Quickstart for GitHub Copilot. <https://docs.github.com/en/copilot/get-started/quickstart>
- GitHub Changelog. (2026, July 8). GitHub Copilot in Visual Studio Code, June 2026 releases.
<https://github.blog/changelog/2026-07-08-github-copilot-in-visual-studio-code-june-2026-releases/>
- Mathieu, E., Giattino, C., Samborska, V., & Roser, M. (2023). Share of professional software developers using AI tools [Data set]. *Our World in Data*.
<https://ourworldindata.org/grapher/share-of-professional-software-developers-using-ai-tools>
- McKinsey & Company. (2023, June 27). Unleashing developer productivity with generative AI.
<https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/unleashing-developer-productivity-with-generative-ai>
- MDN Web Docs. (n.d.). Fetch API. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- OpenAI. (2026a). Using Codex with your ChatGPT plan.
<https://help.openai.com/en/articles/11369540-using-codex-with-your-chatgpt-plan>
- OpenAI. (2026b). Codex in ChatGPT. <https://openai.com/th-TH/codex/>

- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. *Communications of the ACM*.
<https://doi.org/10.1145/3610721>
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv:2302.06590*.
<https://arxiv.org/abs/2302.06590>
- PHP Group. (n.d.). Prepared statements (mysqli). *PHP Manual*.
<https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php>
- Sandoval, G., Pearce, H., Nys, T., Karri, R., Garg, S., & Dolan-Gavitt, B. (2023). Lost at C: A user study on the security implications of large language model code assistants. In *Proceedings of the 32nd USENIX Security Symposium* (pp. 2205–2222). USENIX Association.
- Stack Overflow. (2025). 2025 Stack Overflow Developer Survey: AI.
<https://survey.stackoverflow.co/2025/ai>
- Stray, V., Brandtzæg, E. G., Wivestad, V. T., Barbala, A., & Moe, N. B. (2026). Developer productivity with and without GitHub Copilot: A longitudinal mixed-methods case study.
arXiv:2509.20353. <https://arxiv.org/abs/2509.20353>
- Sun, D., Boudouaia, A., Zhu, C., & Li, Y. (2024). Would ChatGPT-facilitated programming mode impact college students' programming behaviors, performances, and perceptions? An empirical study. *International Journal of Educational Technology in Higher Education*, 21(14). <https://doi.org/10.1186/s41239-024-00446-5>
- Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts* (pp. 1–7). ACM.
<https://doi.org/10.1145/3491101.3519665>
- Ziegler, A., Kalliamvakou, E., Li, X. A., Rice, A., Rifkin, D., Simister, S., Sittampalam, G., & Aftandilian, E. (2024). Measuring GitHub Copilot's impact on productivity. *Communications of the ACM*, 67(3), 54–63. <https://doi.org/10.1145/3633453>

ภาคผนวก ก คู่มือการติดตั้งและใช้งานระบบ

การติดตั้ง

1. ติดตั้ง XAMPP เปิด XAMPP Control Panel แล้วกด Start ที่ Apache และ MySQL
2. คัดลอกโฟลเดอร์โปรเจกต์ไปไว้ที่ C:\xampp\htdocs\thawirod-pos
3. เปิด <http://localhost/phpmyadmin> เลือกแท็บ Import เลือกไฟล์ database.sql แล้วกด Go
4. เปิดเว็บเบราว์เซอร์ไปที่ <http://localhost/thawirod-pos/>
5. หากตั้งรหัสผ่าน MySQL ไว้ ให้แก้ตัวแปร \$pass ในไฟล์ api/get_products.php และ api/update_stock.php

การใช้งาน

1. คลิกการ์ดสินค้าเพื่อสแกนสินค้าเข้าตะกร้า (คลิกซ้ำเพื่อเพิ่มจำนวน)
2. หากต้องการลบรายการ กดปุ่ม × สีแดงที่รายการนั้นในตะกร้า
3. ตรวจสอบยอดชำระสุทธิ แล้วกดปุ่ม "ชำระเงิน"
4. เลือกวิธีชำระเงิน กด "พิมพ์ใบเสร็จ" หากต้องการพิมพ์
5. กด "ปิดหน้าต่าง / เริ่มบิลใหม่" ระบบจะตัดสต็อกในฐานข้อมูลและพร้อมขายบิลถัดไป

การทดสอบตรรกะการคำนวณ

เปิด Terminal ที่โฟลเดอร์โปรเจกต์ แล้วพิมพ์คำสั่ง `node tests/test_logic.mjs` ระบบจะแสดงผลการทดสอบแต่ละกรณีและสรุปจำนวนที่ผ่าน